GUIA VISUAL 072

DEPURAÇÃO DE SCRIPTS SET -X E SHELLCHECK

*"Funciona quando eu rodo na mão." O script de rotação de logs foi testado dezenas de vezes e sempre terminou certo. Posto no `cron` para as 3h, falhou na primeira noite: `line 14: jq: command not found`. O `jq` **estava** instalado — o `which` respondia `/usr/local/bin/jq`. O que mudou não foi o script: foi o **ambiente**. Depurar shell é quase sempre isto — descobrir **qual suposição silenciosa** deixou de valer.*

072
GUIAS LINUX

AS QUATRO CAMADAS · E O AMBIENTE QUE DERRUBOU O SCRIPT DA ABERTURA

1 · NÃO EXECUTA — SEGUNDOS
erro de sintaxe, aspas faltando, variável atribuída e nunca usada
`shellcheck · bash -n · shfmt`

2 · EXECUTA E VÊ — MINUTOS
expansão inesperada, argumento partido, caminho errado
`set -x · PS4 · bash -x`

3 · REGISTRA — HORAS
falha intermitente, erro só em produção, ordem dos eventos
`log em arquivo · BASH_XTRACEFD`

4 · ISOLA — DIAS
bug que só aparece com dados reais ou sob concorrência
`bisseção · bats · trap DEBUG`

SEU TERMINAL — INTERATIVO, DE LOGIN

- `PATH` completo, com `/usr/local/bin` e `/opt`
- lê `/etc/profile` e o perfil do usuário: **aliases e funções existem**
- **TTY presente** — cor faz sentido; `stdin` é o teclado
- `LANG=pt_BR.UTF-8`; `HOME` e `USER` são os seus

SOB CRON OU TEMPORIZADOR DO SYSTEMD

- `PATH` mínimo: `/usr/bin:/bin` — e o `jq` some
- **não lê nenhum arquivo de perfil**: alias não existe, função só se o script definir ou fizer `source`
- **sem TTY**: nada de cor nem de prompt; `stdin` fechado ou `/dev/null`
- `LANG=C` ou vazio; `HOME` pode ser `/` ou o do serviço

A regra do funil: suba de camada só quando a de baixo não resolveu. Pular direto para o rastro de um script de 400 linhas quando o ShellCheck acharia o defeito em dois segundos é o desperdício mais comum de tempo em depuração de shell. A passagem da camada 2 para a 3 é a mais importante: um defeito que **não reproduz** no terminal não vai ser resolvido olhando mais forte para o terminal — ele precisa de **registro persistente do ambiente em que acontece**. **As cinco defesas contra o "funciona na minha máquina":** (1) caminho absoluto, ou `PATH` declarado no topo do script; (2) nada de alias — e função só existe se o próprio script a carregar; (3) `[[ -t 1 ]]` antes de emitir cor, para não encher o log de sequências de escape; (4) `export LC_ALL=C` quando a saída for ordenada ou comparada; (5) **reproduza o ambiente antes de acusar o script**. Fora do funil ainda existe o clássico do arquivo vindo do Windows: `line 2: $'\r': command not found`, diagnosticado com `file script.sh` ("with CRLF line terminators") ou `cat -A`, e curado com `dos2unix` ou `sed -i 's/\r$//'`.

SHELLCHECK ANTES DE RODAR · SET -X DURANTE

kali@kali: ~/lab-depuracao

```
$ shellcheck rotacionar.sh
In rotacionar.sh line 8:
for arq in $(ls /var/log/*.log); do
           ^-- SC2045: Iterating over ls output is fragile.
                ^-- SC2046: Quote this to prevent word splitting.
# o rastro mostra o que o SHELL entendeu, não o que você escreveu
$ bash -x -c 'arq="meu relatorio.txt"; rm -f $arq'
+ arq='meu relatorio.txt'
+ rm -f meu relatorio.txt   # DOIS argumentos!
# PS4 diz onde cada linha do rastro aconteceu
$ export PS4='+ ${BASH_SOURCE##*/}:${LINENO}:${FUNCNAME[0]:-main}(): '
+ backup.sh:24:validar(): mountpoint -q /mnt/backup
# rastro fora da saída, para ler com calma depois
$ exec 9>>/tmp/rastro.log; export BASH_XTRACEFD=9; set -x
# reproduza o cron sem esperar até as 3h da manhã
$ env -i HOME="$HOME" PATH=/usr/bin:/bin \
    /bin/bash --noprofile --norc ./script.sh </dev/null
shellcheck NAO encontrado neste PATH
```

Ligue o rastro só onde ele serve. No script inteiro produz rastro demais para ler: use `set -x; trecho; set +x` em volta do bloco suspeito, ou deixe uma chave de ambiente — `[[ -n "${DEBUG:-}" ]] && set -x` — e rode `DEBUG=1 ./script.sh` **sem editar nada**. Para o defeito que resiste: **bisseção** (um `exit` no meio; funcionou? está na outra metade — cinco iterações localizam num script de 300 linhas), `trap DEBUG` como passo a passo, ou `bats` para virar teste. E **escreva o teste que reproduz o defeito antes de corrigi-lo**: é o que impede que ele volte.

OS AVISOS QUE SÃO CORREÇÃO, NÃO ESTILO

SC2086	expansão **sem aspas** — quebra em espaços e sofre globbing
SC2046	substituição de comando sem aspas — mesma coisa, forma mais perigosa
SC2164	`cd` sem tratar falha; use `cd x || exit`
SC2155	`local v=$(cmd)` — o status é o do `local`, sempre 0, e a falha de `cmd` **some**. Declare numa linha, atribua na seguinte
SC2181	`if [ $? -eq 0 ]` é frágil: qualquer comando entre as duas linhas destrói o `$?`. Teste o comando direto: `if grep -q p arq; then`
SC2115	`rm -rf "$DIR/build"` sem garantia de que `DIR` tem valor. Use `"${DIR:?DIR nao definida}"`
SC2034	variável atribuída e nunca usada — quase sempre erro de digitação
SC2064	aspas **duplas** no `trap`: a expansão acontece no registro, não no disparo
SC1090	`source "$arq"` com caminho variável não pode ser analisado

SILENCIAR COM RESPONSABILIDADE

A diretiva vai **na linha imediatamente anterior** e **com justificativa**: `# shellcheck disable=SC2086  # a expansão sem aspas é intencional: OPCOES é lista de flags`. No topo do arquivo ela vale para o arquivo inteiro — raramente o que se quer. `disable` **sem comentário é dívida:** seis meses depois ninguém sabe se era intencional ou se alguém só queria calar o analisador. Úteis também: `# shellcheck shell=bash` (biblioteca sem shebang) e `# shellcheck source=lib/comum.sh` (resolve o `SC1090`).

PLUGUE NO FLUXO

`shellcheck -S warning $(find . -name '*.sh')` varre o repositório; `-f gcc` dá `arquivo:linha:coluna:` para a integração contínua; e o gancho de pré-commit é `git diff --cached --name-only --diff-filter=ACM | grep '\.sh$' | xargs -r shellcheck`. Companheiro natural é o `shfmt -d -i 4 -ci` (mostra o diff) / `-w` (aplica): **com indentação regular, um `fi` no lugar errado salta aos olhos**. E antes de qualquer script que apague, formate ou reinicie algo, `bash -n` — um `fi` faltando em 300 linhas só aparece quando a execução chega lá, e aí metade do estrago já foi feita.

O `SC2115` é o aviso que evita apagar o sistema. `rm -rf "$DIR/build"` parece seguro por ter aspas — mas se `DIR` estiver vazia (nome digitado errado, ou variável definida só num ramo do `if`), o comando executado é `rm -rf "/build"`, e com privilégio suficiente ele apaga o que encontrar na raiz. A forma que **falha em vez de destruir** é `rm -rf "${DIR:?DIR nao definida}/build"`. Antes de rodar qualquer script novo que apague coisas, use as três camadas baratas na ordem: `shellcheck`, `bash -n` e **o rastro com o `rm` trocado por `echo rm`** — o `set -x` mostra o caminho **já expandido**, que é exatamente a informação que faltava. E ensaie em VM descartável, nunca na máquina de trabalho.

RESUMO RÁPIDO

- ✓ **A causa mais comum de "funciona na minha máquina" é o ambiente**, não o código: `PATH` mínimo, nenhum perfil lido, sem TTY e localidade `C` sob `cron` e systemd.
- ✓ **Reproduza antes de acusar o script:** `env -i HOME="$HOME" PATH=/usr/bin:/bin /bin/bash --noprofile --norc ./script.sh` .
- ✓ **O ShellCheck é a ferramenta de melhor retorno por segundo investido.** `SC2086` , `SC2155` , `SC2181` e `SC2115` descrevem defeitos reais, não estilo.
- ✓ `local x=$(cmd)` **esconde a falha de** `cmd` — o status é o do `local` . Declare numa linha, atribua na seguinte.
- ✓ `set -x` **mostra o comando depois das expansões**: é assim que se enxerga o argumento partido pela falta de aspas. Em volta do trecho suspeito, não no script inteiro.
- ✓ `PS4` **transforma o rastro em diagnóstico** — arquivo, linha e função em cada linha — e `BASH_XTRACEFD` tira o rastro da saída, mantendo o `stdout` limpo para quem consome o resultado.
- ✓ **Silenciar aviso exige justificativa escrita** na linha anterior; `disable` sem comentário é dívida.
- ✓ **Suba de camada só quando a de baixo não resolveu** — estática, rastro, registro, isolamento — e escreva o teste que reproduz o defeito antes de corrigi-lo.

NO KALI

Ferramenta de segurança em script quase sempre roda sob `cron` **ou temporizador do systemd** — coleta periódica, varredura agendada, verificação de integridade caem exatamente na armadilha da abertura. **Declarar** `PATH` **no topo do script é obrigatório no Kali**, onde muita ferramenta vive em `/usr/share/<ferramenta>` com um invólucro em `/usr/bin` , e o que se instala à mão fica em `/opt` . **Cuidado: o rastro do** `set -x` **vaza segredo.** Se o script manipula senha, token ou chave, o rastro imprime o **valor já expandido** — e se ele vai para log persistente, a credencial fica gravada em claro. Desligue em volta do trecho sensível (`set +x` antes, `set -x` depois) e nunca passe segredo por linha de comando, onde ele também apareceria no `ps` . **Um** `SC2086` **num script que monta a lista de alvos é a diferença entre varrer** `10.0.0.1` **e varrer dois hosts** porque a variável trazia dois valores: estourar o escopo por falta de aspas não é atenuante nenhum diante da Lei 12.737/2012, e a conferência estática é barata. `[[ -t 1 ]]` **antes de colorir:** ferramenta do Kali produz muita saída colorida, e um script que a captura com `$( )` recebe as sequências de escape junto — a comparação de string falha por um motivo invisível. Desligue a cor na origem (`--no-color` , `NO_COLOR=1`) ou limpe com `sed -r 's/\x1b\[[0-9;]*m//g'` . E **log com marca de tempo é evidência**: num teste autorizado, o registro de quando cada comando foi disparado é o que separa a sua atividade da de terceiros no mesmo período — `PS4` com `\D{%F %T}` e rastro em arquivo dedicado viram auditoria da própria execução. **Família RHEL:** o `shellcheck` está no EPEL (`dnf install epel-release` antes); o `cron` do usuário pode rodar com `SHELL=/bin/sh` , fazendo `[[ ]]` e arrays falharem com mensagens confusas (declare `SHELL=/bin/bash` no topo do `crontab`); e o SELinux em `enforcing` produz "Permission denied" com permissões POSIX corretas — esse diagnóstico está no `ausearch -m avc -ts recent` , não no `set -x` .